— 포트폴리오 · 2026

Kakhkhorov Shokhjakhon (Sean)

풀스택 개발자 ◆ AI 서비스 개발 · DevOps

웹 화면, API, Linux 서버 배포를 담당해 온 실무 경력 3년 이상의 풀스택 개발자입니다. TypeScript, React·Next.js, NestJS, Python·FastAPI, MongoDB, PostgreSQL을 활용해 크리에이터 탐색 플랫폼, 선적 서류 검토 도구, 출처 기반 AI 어시스턴트 프로젝트를 개발하고 있습니다.

엔드 투 엔드 개발

데이터 모델, API, 인증, 실시간 기능부터 UI까지

배포와 운영

Docker, Nginx, DNS, TLS와 GitHub Actions CI

AI 적용

문서 추출, 하이브리드 검색, 출처에 근거한 답변

TypeScript · React · Next.js · Node.js · NestJS · Express · GraphQL · Socket.io · Python · FastAPI · MongoDB · PostgreSQL · Redis · Docker · Nginx · GitHub Actions · Gemini API

대한민국 수원 · kakhkhorov97@gmail.com · github.com/kakhkhorov97



라이브 프로젝트 · creatorhub.site

— 목차

주요 프로젝트와 경력

01 자기소개 — 배경과 일하는 방식 — 03쪽

02 경력 — IT Local, Atlantis Investment — 04쪽

03 기술 — 프로젝트별로 정리한 기술 스택 — 06쪽

04 CreatorHub — 서비스 중 · 개인 프로젝트 · 사례 연구 — 07쪽

05 CreatorHub 제품 — 화면, 문의, AI 파인더 — 11쪽

06 CreatorHub 엔지니어링 — 아키텍처와 데이터 모델 — 14쪽

07 CargoCheck AI — 개발 프로젝트 · 서류 검토 — 17쪽

08 Lexara — 공동 프로젝트 · 개발 참여 — 22쪽

09 Korea Visa Assistant — 개발 중 · 검색 기반 — 23쪽

10 Sofiya Bookstore — 이전 프로젝트 — 25쪽

11 연락처 — 이메일, 전화번호, 링크 — 26쪽

— 자기소개

코드부터 운영까지, 풀스택 개발.

Express와 MongoDB API를 만드는 백엔드 개발자로 시작해, 2023년 6월부터 2026년 1월까지 IT Local에서 풀스택 개발자로 일했습니다. 4인 이상 개발팀의 일원으로 20개 이상 고객사를 맡았고, 데이터베이스 설계와 인증부터 배포, DNS, 장애 대응까지 담당하면서 코드를 어떻게 작성하는지뿐 아니라 배포 후 어떻게 동작하는지까지 신경 쓰게 되었습니다.

주요 프로젝트: 혼자 개발·배포해 **creatorhub.site**에서 운영 중인 크리에이터 탐색 플랫폼 **CreatorHub**, 상업송장과 포장명세서를 대조하는 개발 단계 도구 **CargoCheck AI**, 다른 개발자와 함께 개발에 참여한 문서 질의응답 플랫폼 **Lexara**, 그리고 개발 중인 검색 기반 어시스턴트 **Korea Visa Assistant**입니다.

Claude Code, Codex 같은 코딩 에이전트를 명문화된 저장소 규칙 안에서 활용합니다. 모든 변경은 제가 직접 승인하고, 병합 전에는 자동 검사를 거칩니다.

안정성
모니터링, 로그 분석, 신속한 장애 대응

보안
인증, 입력값 검증, 코드 리뷰

문서화
API와 배포 절차 문서화

AI 워크플로
검토 단계를 갖춘 코딩 에이전트



Kakhkhorov Shokhjakhon (Sean)

풀스택 개발자
AI 서비스 개발 · DevOps

— 경력

풀스택 웹 개발자
IT Local · 2023.06 – 2026.01

백엔드 개발자
Atlantis Investment · 2022.05 – 2022.12

— 거주지

대한민국 경기도 수원시

— 연락처

github.com/kakhkhorov97
kakhkhorov97@gmail.com

— 경력

근무 경력

풀스택 웹 개발자

2023.06 - 2026.01

IT Local · 우즈베키스탄

웹 개발, 서버·컴퓨터 조립, 네트워크 구성, CCTV 설치 서비스를 제공하는 IT 서비스 회사.

- 4인 이상 개발팀에서 건설·유통·요식·기업/공공기관 등 20개 이상 고객사 웹사이트 개발·유지보수
- 인증, 권한 부여, 입력값 검증을 갖춘 API와 서버 측 비즈니스 로직 개발
- 데이터베이스 스키마 설계, 인덱싱·쿼리 최적화로 페이지·API 응답 속도 개선
- Ubuntu 서버 5~15대 구축 및 SSH 접근·방화벽 규칙 등 보안 설정
- Nginx 뒤에서 PM2와 Docker로 Node.js 서비스 운영, Cloudflare로 DNS·TLS 관리
- 수동 배포를 CI 기반 자동 배포로 전환
- 애플리케이션 상태 모니터링, 로그 분석, 장애 및 성능 문제 해결
- 팀원 코드 리뷰, API와 배포 절차 문서화
- React, Bootstrap, Tailwind CSS로 반응형 페이지 개발
- 퇴사 전 후임자 온보딩 및 모든 시스템 인수인계

백엔드 개발자

2022.05 - 2022.12

Atlantis Investment · 우즈베키스탄

대학 학업과 병행.

- Express.js로 프론트엔드와 백엔드를 잇는 RESTful API 구축
- Mongoose를 활용한 MongoDB 데이터베이스 설계·유지 및 데이터 모델링
- MVC 아키텍처로 관심사를 분리해 유지보수와 확장이 쉬운 코드 구성
- 세션과 JWT 기반 회원가입·로그인 구현, bcryptjs로 비밀번호 해싱
- Multer로 이미지·문서 파일 업로드 엔드포인트 구축
- 요청 데이터 검증과 일관된 에러 처리 및 상태 코드 적용
- 검색·필터링·페이지네이션을 위한 MongoDB 쿼리와 인덱스 작성
- Postman으로 엔드포인트 테스트·문서화, 프론트엔드와 요청·응답 형식 협의

— 경력 · IT LOCAL

고객사 웹사이트, 개발부터 서버까지.

IT Local은 다른 기업의 웹사이트를 개발하고 운영합니다. 4인 이상 개발팀에서 첫 페이지부터 그 페이지가 돌아가는 서버까지 전 과정을 맡았습니다.

20개 이상 고객사
건설, 유통, 요식, 기업 및 공공기관

→

4인 이상 개발팀
백엔드, 프론트엔드, 서버, 코드 리뷰

→

CI 배포
수동 배포 단계를 자동화

→

Ubuntu 서버 5~15대
Nginx, PM2, Docker, Cloudflare

— 직접 해결한 문제 세 가지

수동 배포 → CI

문제. 배포를 수작업으로 했습니다. 서버마다 SSH로 접속해 코드를 받고, 설치하고, 재시작했습니다. 5–15대 서버에 걸친 여러 고객사 사이트에서 한 단계만 빠져도 사이트가 멈출 수 있었고, 절차를 아는 사람만 배포할 수 있었습니다.

한 일. 배포 절차를 문서로 정리한 뒤, 매번 같은 방식으로 설치·빌드·재시작하는 CI 파이프라인으로 옮겼습니다.

결과. 모든 배포가 검토된 같은 절차를 따르게 되었고, 다른 개발자도 저 없이 배포할 수 있게 되었습니다.

고객사 사이트 장애 대응

문제. 고객사 사이트가 가끔 다운되거나 Nginx 뒤에서 오류를 반환했습니다.

한 일. 장애가 날 때마다 Nginx, PM2, 애플리케이션 로그를 따라가 실패 지점을 찾아 근본 문제를 수정하고, PM2가 서비스를 자동으로 재시작하도록 설정했습니다.

결과. 사이트를 복구했고, 해결 과정을 문서로 남겨 팀이 다음에 같은 절차를 따를 수 있게 했습니다.

데이터 증가로 느려진 페이지

문제. 고객 데이터가 늘면서 목록·검색 페이지가 느려졌습니다.

한 일. 느린 쿼리를 확인해 필터와 정렬 순서에 맞는 인덱스를 추가하고, 전체를 불러오는 대신 페이지네이션을 적용했습니다.

결과. 쿼리가 새 인덱스를 사용하게 되어, 데이터가 계속 늘어도 목록·검색 페이지가 원활하게 동작했습니다. 개선 전후 시간은 따로 기록하지 않았습니다.

고객사 사이트는 IT Local 고객의 소유이므로 여기에 싣지 않았습니다.

— 기술 및 도구

기술 스택

핵심 · 매일 사용 사용 경험 있음

프론트엔드

React · Next.js · TypeScript · JavaScript · Apollo Client · Redux Toolkit · MUI · Tailwind CSS · SCSS · HTML · CSS · jQuery · Axios · TanStack Query · React Router · Socket.IO Client · Framer Motion · Three.js · Swiper · Anime.js · Typed.js · TUI Editor · Bootstrap · Chakra UI · shadcn/ui · Figma

백엔드

Node.js · NestJS · Express · GraphQL · REST API · Socket.io · JWT & OAuth · Python · FastAPI · Zod · 역할 기반 접근 제어 · EJS · bcryptjs · Multer

데이터베이스

MongoDB · Mongoose · 인덱싱·쿼리 최적화 · 트랜잭션 · PostgreSQL · Redis · BullMQ · Chroma · MySQL · MongoDB Compass

DevOps

Docker · Docker Compose · Nginx · GitHub Actions · Cloudflare · DNS·TLS · PM2 · 방화벽 설정 · Git & GitHub · GitHub Pages · Render

AI·검색

Gemini API · OpenAI API · 하이브리드 검색 (BM25 + 벡터) · OCR (Tesseract, Poppler) · 구조화 추출 · Kiwi 형태소 분석기 · RRF · hit@k · MRR 평가 도구 · Claude Code · Codex · Cursor

테스트 및 도구

Jest · Supertest · ESLint · Prettier · pytest · 통합 테스트 · API 계약 테스트 · PostHog · Postman · npm

사용한 프로젝트: CreatorHub에서 NestJS, GraphQL, MongoDB, Docker · CargoCheck AI에서 NestJS, PostgreSQL, Redis, OCR, Gemini · Korea Visa Assistant에서 Python, FastAPI, Kiwi, BM25, Chroma · Lexara 제품 스택(공동 프로젝트)의 FastAPI, PostgreSQL, pgvector.

— 주요 프로젝트 · 01

CreatorHub.

개인 프로젝트 (2026.04 – 현재): 프론트엔드, 백엔드, 인프라를 직접 설계·개발·배포했습니다.

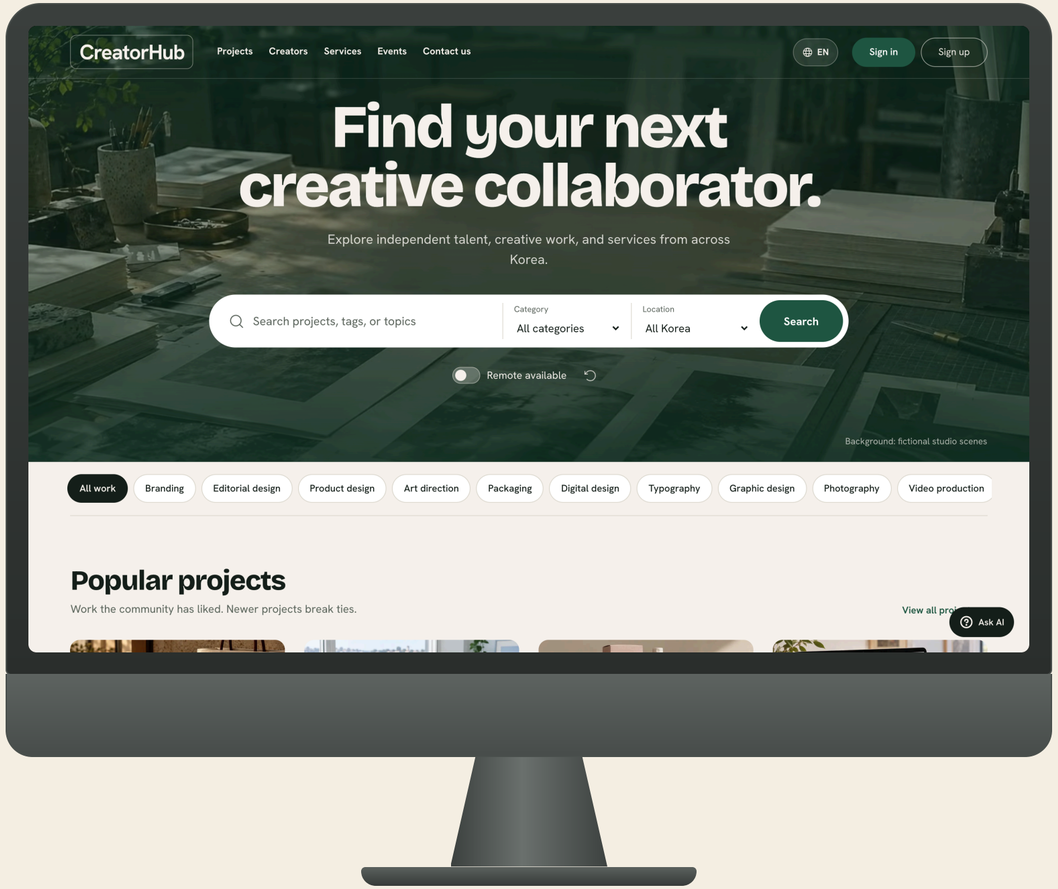
방문자가 한국 각지의 독립 크리에이터를 찾고, 그들의 프로젝트와 서비스를 살펴보며, 비공개 프로젝트 문의를 보낼 수 있는 포트폴리오 탐색 플랫폼입니다.

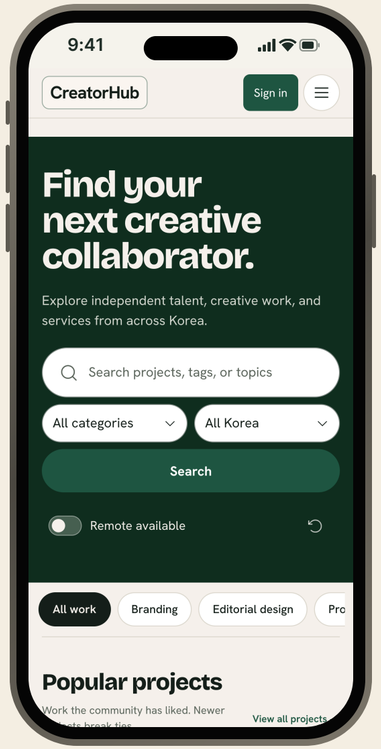
- 카테고리, 지역, 원격 근무로 검색
- 관리자 승인을 거치는 크리에이터 워크스페이스
- 실시간 채팅이 가능한 비공개 문의

프론트엔드	Next.js · React · TypeScript
백엔드	NestJS · GraphQL · Socket.io
데이터	MongoDB · Mongoose
인프라	Docker · Nginx · GitHub Actions
AI	Gemini API

사이트 방문 ↗

저장소: CreatorHub · CreatorHub-next





— CREATORHUB · 사례 연구

한 번 클릭, 하나의 문의: 재시도에 안전한 요청

문제. 더블 클릭이나 끊긴 연결 때문에 같은 문의나 채팅 메시지가 두 번 전송될 수 있었습니다. 서버는 이를 새 요청 두 개로 보고 둘 다 저장해, 크리에이터에게 알림도 두 번 보낼 수 있었습니다.

1

폼마다 하나의 키

브라우저가 폼마다 키를 하나 만들고, 재시도할 때마다 같은 키를 다시 씁니다

`crypto.randomUUID()`

→

2

먼저 확인

트랜잭션 안에서, 이미 알려진 키라면 저장된 레코드를 그대로 돌려줍니다

`findOne(clientId, key)`

→

3

최후 방어선인 인덱스

두 요청이 동시에 오면 유니크 인덱스가 두 번째 삽입을 거부합니다

`partial unique index`

→

✓

하나의 레코드

밀린 요청은 첫 번째 레코드로 응답하므로 사용자는 결과를 하나만 봅니다

`uq_inquiry_request`

같은 키, 다른 내용

같은 메시지 키를 다른 내용으로 다시 쓰면, 기존 메시지를 조용히 유지하는 대신 충돌 오류를 돌려줍니다.

모든 변경에 버전

모든 상태 변경은 기준이 된 버전을 함께 보내므로, 오래된 화면이 더 새로운 상태를 덮어쓸 수 없습니다.

검증

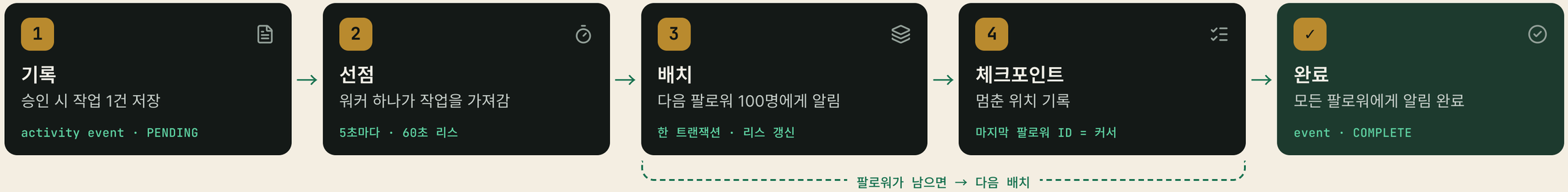
통합 테스트에서 같은 문의를 두 번 제출하고 같은 키로 메시지를 동시에 보냅니다. 결과는 정확히 이벤트 1건, 메시지 1건, 알림 1건입니다.

좋아요, 저장, 팔로우, 댓글 재시도에도 같은 패턴을 적용했습니다.

— CREATORHUB · 사례 연구

워커가 재시작되어도 유지되는 팔로워 알림

문제. 크리에이터가 새 작업을 공개하면 모든 팔로워에게 알림이 갑니다. 워커가 중간에 멈추면 일부 팔로워는 알림을 두 번 받거나 아예 받지 못할 수 있습니다.



중단이나 오류가 나면? 오류가 나면 30초 후 재시도하도록 작업을 풀어 주고, 워커가 멈추면 60초 리스가 만료됩니다. 어느 경우든 다음 작업은 저장된 커서부터 이어갑니다.

중복 방지

알림은 팔로워·이벤트마다 하나뿐이라, 배치가 반복돼도 같은 사람에게 두 번 알리지 않습니다.

알려진 한계

MongoDB 폴링으로 별도 큐 서비스는 필요 없지만 몇 초의 지연이 생깁니다. 아직 재시도 횟수 제한이 없어, 다음 단계로 일정 횟수 후 중단하고 관리자에게 표시할 계획입니다.

검증

통합 테스트로 롤백된 배치가 중복 없이 재시도되는지, 첫 공개 때만 팔로워에게 알리는지 확인합니다.

범위: 앱 내 알림만 해당하며, 팔로워·이벤트당 최대 1건입니다. 이메일이나 푸시는 보내지 않습니다.

— CREATORHUB · 사례 연구

테스트, CI/CD, 롤백

두 저장소 모두 푸시할 때마다 테스트합니다. 배포는 수동으로 시작하는 별도 워크플로를 거칩니다.

— 테스트 범위

백엔드 단위 · Jest
비즈니스 규칙: 문의 상태, 비밀번호, 요청 제한

API 통합 · Jest
일회용 MongoDB 레플리카 셋에서 실제 GraphQL 요청

프론트엔드 · Node.js
세션, 로그인, AI 채팅 라우트, 번역

정적 검사
TypeScript, ESLint, 모든 쿼리를 스키마와 대조

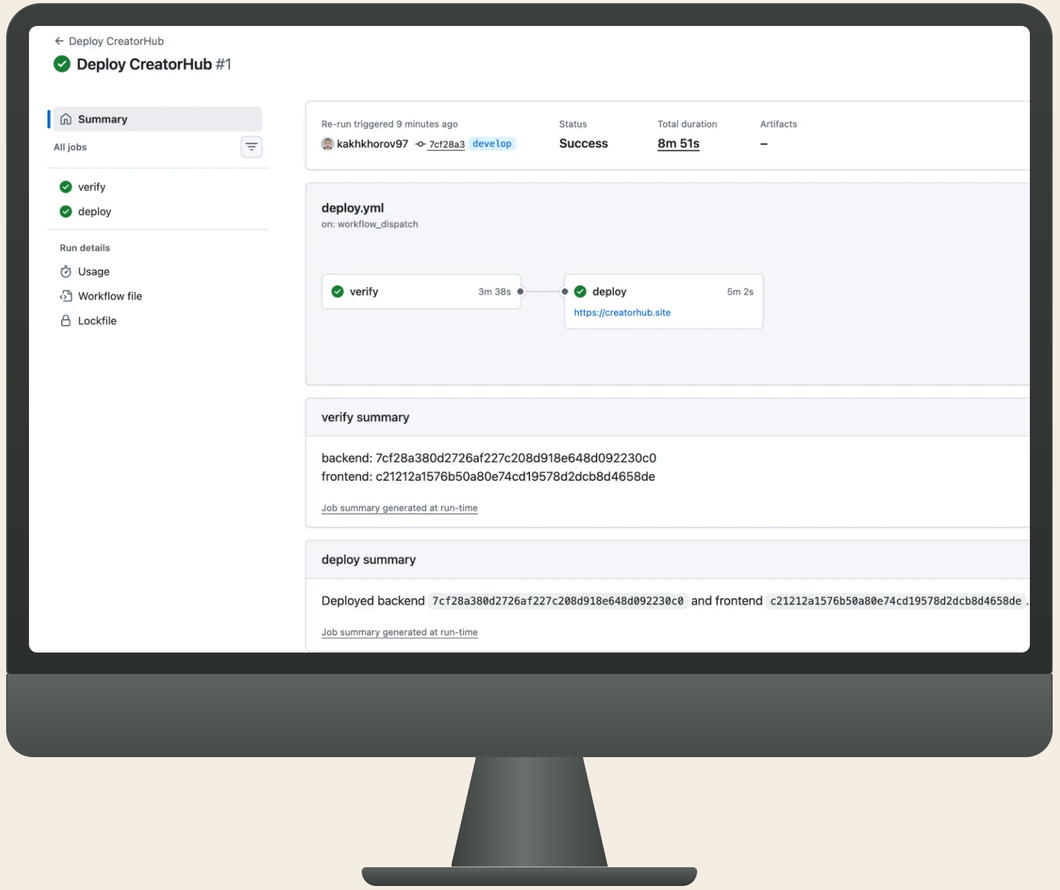
recorded locally · 17 Oct 2026

```
$ backend   jest           ✓ 162/162 tests · 16 suites
$ backend   deploy scripts ✓ 23/23
$ frontend  node --test    ✓ 108/108
$ both      typecheck · lint · contract (119 ops) ✓ passed
= 293 automated checks recorded (162 + 23 + 108)
```

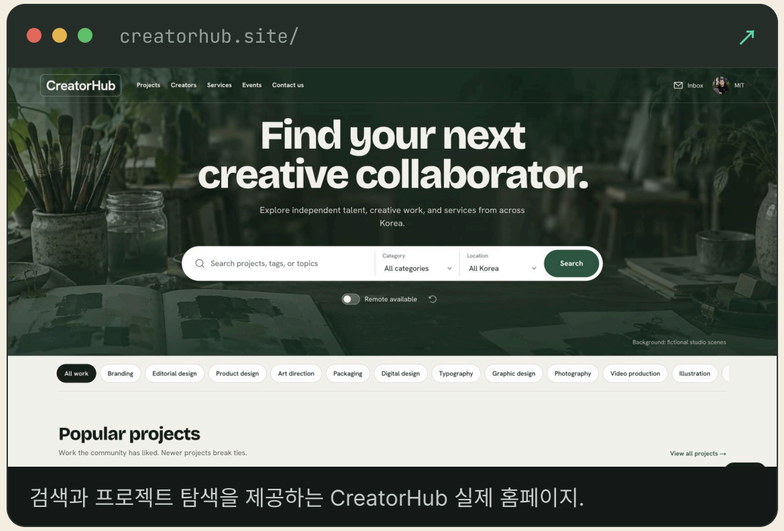
— 배포 과정

1. **develop·main에 푸시할 때마다** 설치, 타입 검사, 린트, 테스트, 빌드를 실행합니다.
2. **배포 워크플로**는 수동으로 시작하며, 백엔드·프론트엔드 커밋 한 쌍에 대해 모든 검사를 다시 실행합니다.
3. **제한된 SSH**로 “deploy”와 커밋 ID 두 개만 받는 배포 전용 계정에 접속합니다.
4. **헬스 체크 후 전환:** 새 이미지는 설정된 최대 180초 안에 정상 상태를 보고해야 하며, 실패하면 이전 이미지로 자동 복원됩니다.

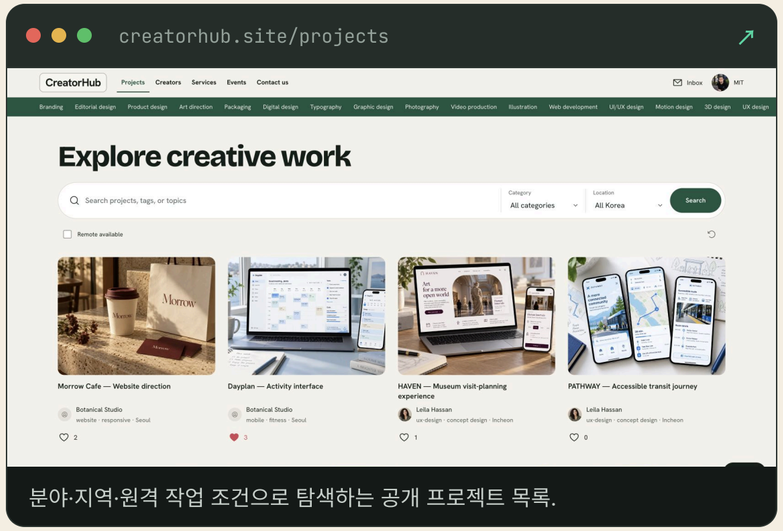
화면: 2026년 10월 18일 첫 Deploy CreatorHub 실행 (검증 3분 38초, 배포 5분 2초). 180초는 설정값이며 측정된 복구 시간이 아닙니다. 데이터베이스 변경은 롤백하지 않습니다.



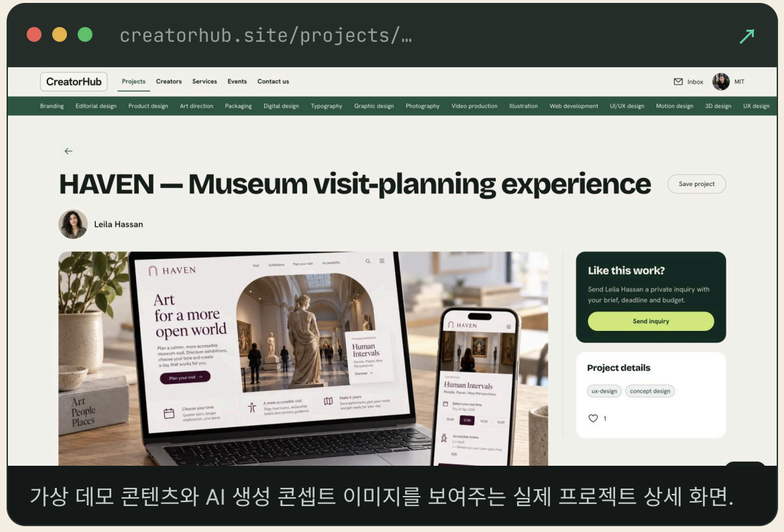
실제 서비스 화면.



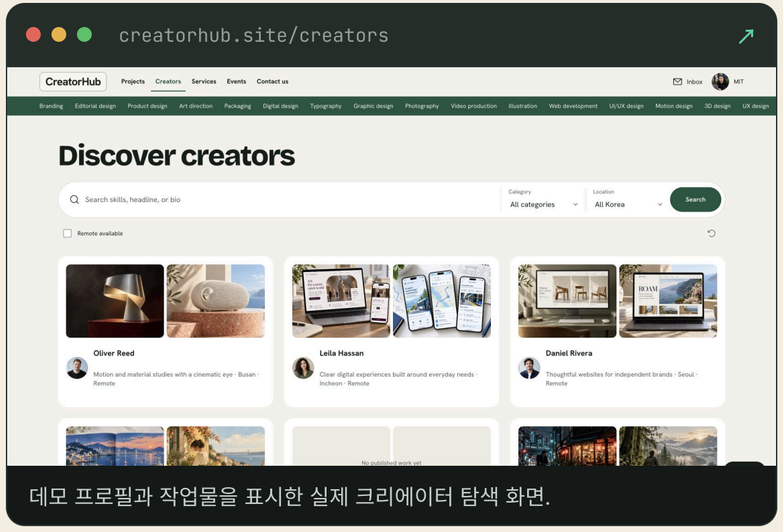
검색과 프로젝트 탐색을 제공하는 CreatorHub 실제 홈페이지.



분야·지역·원격 작업 조건으로 탐색하는 공개 프로젝트 목록.



가상 데모 콘텐츠와 AI 생성 콘셉트 이미지를 보여주는 실제 프로젝트 상세 화면.



데모 프로필과 작업물을 표시한 실제 크리에이터 탐색 화면.

주요 기능

- MongoDB 집계 한 번으로 필터·정렬·페이지 처리, 검색 상태는 URL에 유지
- 프로필, 고정가 또는 별도 견적 서비스, 관리자가 관리하는 이벤트
- 좋아요·저장·댓글·팔로우는 사람과 항목마다 한 번만 기록
- 크리에이터 작업 공간: 관리자가 승인하기 전까지 수정 내용 비공개
- 사용자·권한·검수·사이트 문구를 관리하는 관리자 영역, 서버 측 권한 가드 적용
- 이메일, Google, Kakao, Telegram 로그인 · 영어, 한국어, 우즈베크어 지원

2026년 10월 22일 creatorhub.site에서 캡처. 크리에이터, 프로젝트, 작업물은 가상 데모 콘텐츠입니다.

— CREATORHUB · 워크플로

비공개 문의와 실시간 채팅

클라이언트는 예산과 마감일을 담은 의뢰서를 크리에이터에게 보내고, 비공개 채팅으로 논의합니다.

작성 중
클라이언트가 의뢰서 작성

→

전송됨
크리에이터가 거절 가능

→

논의 중
첫 답장 이후

→

종료
합의, 진행 안 함, 또는 7일간 무응답

기본적으로 비공개

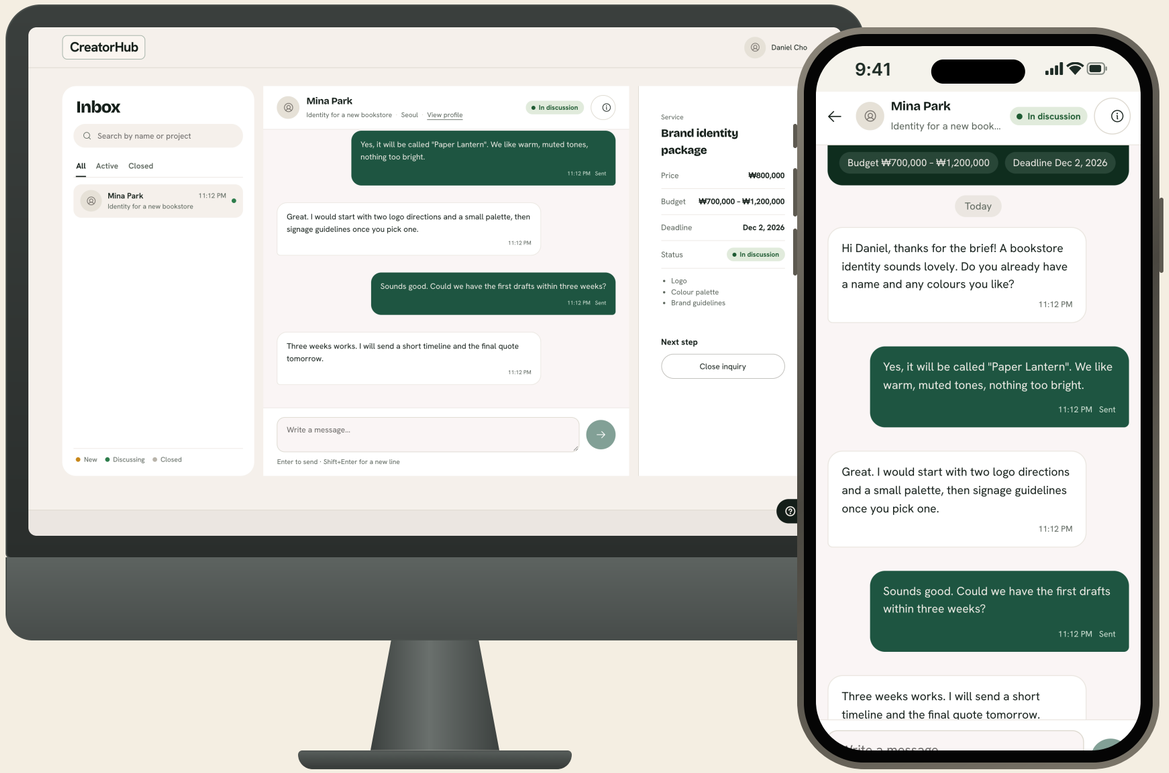
문의는 클라이언트와 크리에이터만 열 수 있습니다. 크리에이터는 작성 중인 초안을 볼 수 없습니다.

중복 방지

더블 클릭이나 네트워크 재시도로 문의나 메시지가 중복 생성되지 않습니다.

실시간 채팅

Socket.io 방도 같은 접근 검사를 거치며, 데이터베이스가 기준 데이터입니다.



현재 버전의 앱을 로컬에서 실행해 가상의 계정과 메시지로 캡처했습니다.

— CREATORHUB · 사례 연구

AI 파인더를 빠르고 안전하게, 비용은 상한 안에서

문제. AI 파인더는 모든 방문자에게 열려 있고, 답변 하나하나가 유료 Gemini 호출입니다. 제한이 없으면 스크립트 하나로 요청이 폭주하고, 느린 조회는 방문자를 기다리게 하며, 모델이 바쁘면 오류가 표시됩니다.

1

라우트 관문

브라우저 출처 확인, 메시지는 최대 1,500자, 방문자당 분당 8개

`origin · size · rate`

→

2

일일 한도

일일 한도가 악용 상황에서도 모델 비용의 상한을 정합니다

`일일 한도 · 기본 200`

→

3

시간 제한이 있는 검색

도움말 최대 3개와 실시간 공개 데이터를 가져오며, 조회는 3.5초 후 중단됩니다

`timeout 3.5 s`

→

✓

장애 시 대체 모델로 전환

모델이 바쁘면 대체 모델로 한 번 재시도하고, 키가 없으면 도움말로 안내합니다

`429 / 500 / 503 → fallback`

근거 기반 답변

낮은 temperature, 최근 대화 최대 8개, 그리고 소스를 지시가 아닌 데이터로 다루도록 지시하는 프롬프트.

키는 서버에만

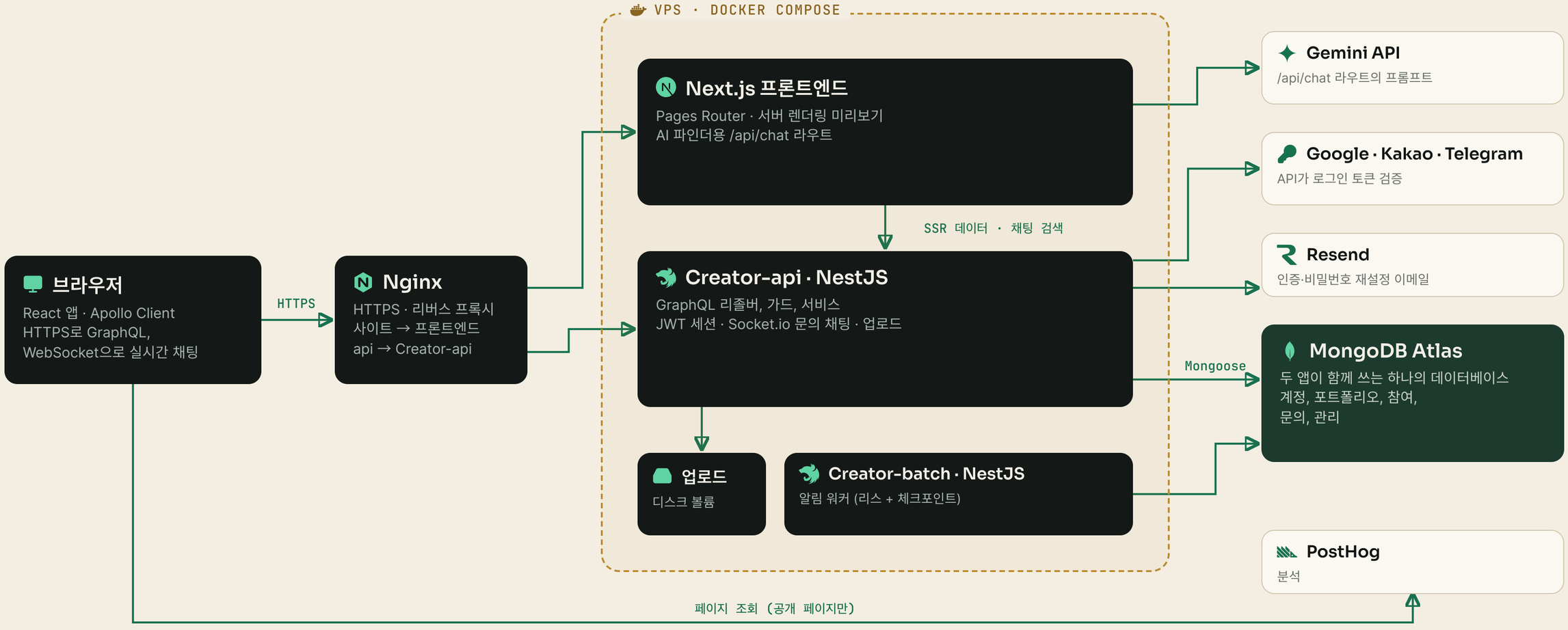
Gemini 키는 서버 라우트에만 있으며, 브라우저로 전달되지 않습니다.

알려진 한계

검색은 키워드 매칭과 GraphQL 조회 한 번으로 이루어지며, 아직 임베딩은 쓰지 않습니다. 다음 단계는 시맨틱 검색입니다.

— CREATORHUB · 아키텍처

세 개의 애플리케이션, 하나의 데이터 모델.



— CREATORHUB · 데이터 모델

데이터 설계 방식.

API와 워커가 함께 쓰는 하나의 MongoDB 데이터베이스에 21개 컬렉션이 있습니다. 참조는 서비스에서 검사하고, 반드시 지켜야 하는 규칙은 유니크 인덱스로 강제합니다.

계정

`users` `authIdentities` `authSessions` `authTokens`

- 로그인 방식마다 별도 레코드라, 한 사람이 이메일·Google·Kakao를 함께 사용 가능
- 리프레시·이메일 토큰은 해시로만 저장되고 자동 만료
- 세션이 레코드로 저장되어, 사용자 정지 시 모든 세션 종료

포트폴리오

`creatorProfiles` `categories` `portfolioProjects` `services`

- 프로필과 크리에이터별 프로젝트에 읽기 쉬운 고유 슬러그
- 소셜 링크와 프로젝트 미디어는 항상 상위 문서와 함께 읽으므로 임베드
- 인덱스는 공개 피드에 맞춰 상태, 최신순으로 구성

참여

`creatorFollows` `projectLikes` `savedProjects` `projectComments` `activityEvents`

- 좋아요·저장·팔로우는 사람·항목당 하나로, 유니크 인덱스로 강제
- 언팔로우는 삭제하지 않고 팔로우를 비활성화
- 활동 이벤트에 워커의 리스와 커서를 저장해 안전하게 전달

문의

`inquiries` `inquiryMessages` `inquiryEvents` `serviceFeedback`

- 서비스 조건은 임베드된 스냅샷으로 문의에 복사
- 재시도 키와 부분 유니크 인덱스로 문의·메시지 중복 방지
- 모든 상태 변경은 번호가 매겨진 이벤트이며, 완료된 문의당 리뷰는 하나

관리

`contentReviews` `notifications` `creativeEvents` `siteContent`

- 프로필·프로젝트·서비스당 검토 대기는 최대 하나
- 알림은 수신자·이벤트마다 하나
- 사이트 문구는 키가 있는 문서 하나라 수정 시 재배포 불필요

문의 관련 필드는 16쪽에서 크게 볼 수 있습니다. 전체 ER 다이어그램은 별도 고해상도 파일(CreatorHub-ER-all-collections-expanded-HQ.png)로 제공합니다.

Inquiries, 필드 단위로

PK 기본 키 FK → 참조 컬렉션 ⬚ 내장 스키마

inquiries

PK	_id	objectId
	clientRequestKey	string
FK	clientId	→ users
FK	creatorId	→ creatorProfiles
FK	serviceId	→ services
	subject	string
	brief	string
	budgetMin	int
	budgetMax	int
	currency	enum
	desiredDeadline	date
	serviceSnapshot	embedded
	status	enum
	outcome	enum
FK	closedById	→ users
	closedAt	date
	submittedAt	date
	lastMessageAt	date
	version	int
	createdAt	date
	updatedAt	date

InquiryServiceSnapshot

embedded in inquiries.serviceSnapshot

	title	string
	pricingType	enum
	startingPrice	int
	currency	enum
	estimatedDays	int
	deliverables	string[]

inquiryMessages

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	senderId	→ users
	clientMessageKey	string
	body	string
	readAt	date
	createdAt	date

inquiryEvents

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	performedById	→ users
	type	enum
	fromStatus	enum
	toStatus	enum
FK	messageId	→ inquiryMessages
	version	int
	createdAt	date

serviceFeedback

PK	_id	objectId
FK	inquiryId	→ inquiries
FK	clientId	→ users
FK	creatorId	→ creatorProfiles
FK	serviceId	→ services
	rating	int
	feedback	string
	createdAt	date
	updatedAt	date

— 주요 프로젝트 · 02

CargoCheck AI

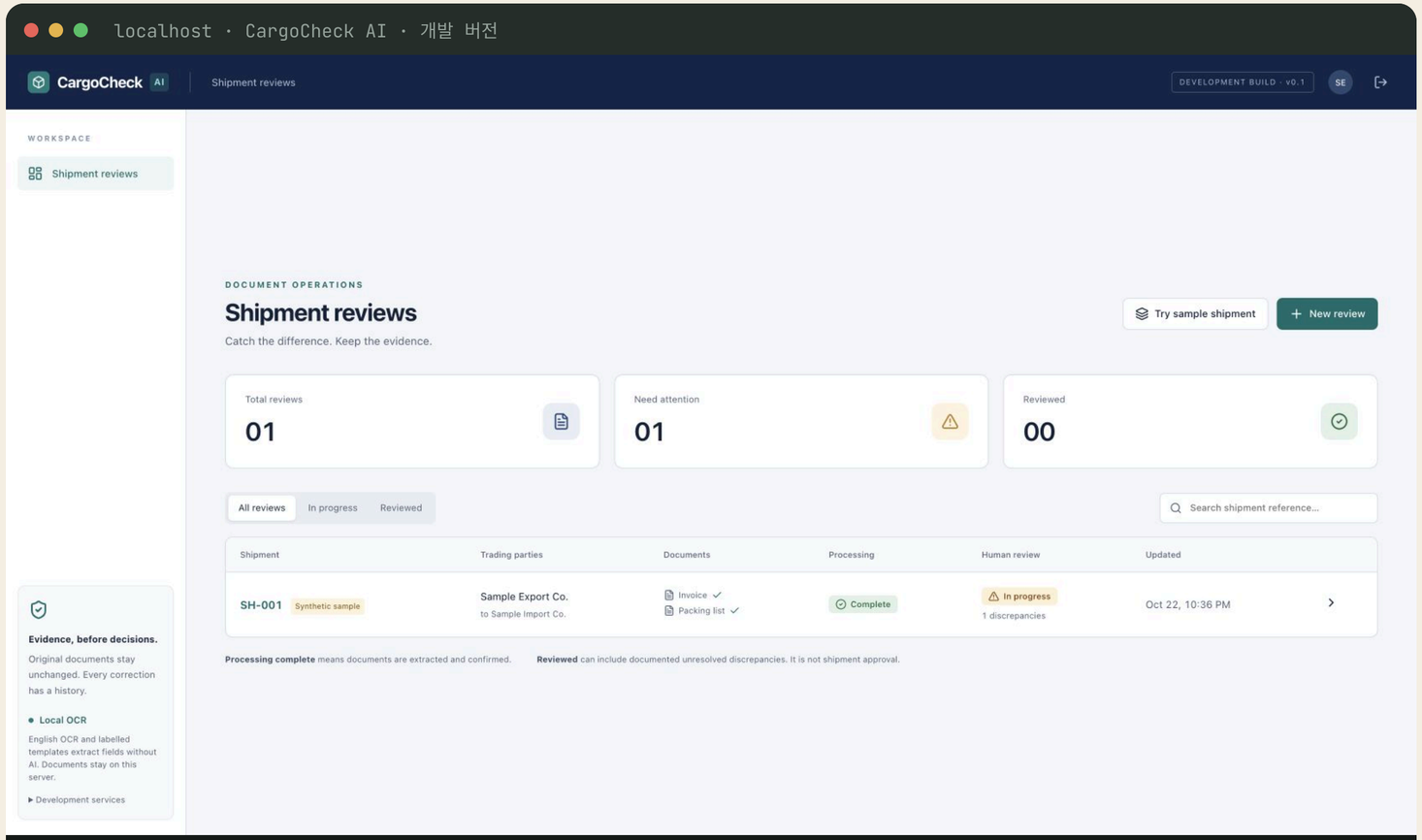
개발 프로젝트이며 정식 출시 버전이 아닙니다. 로컬에서 실행하며 공개 배포는 하지 않았습니다.

상업송장과 포장명세서를 대조하는 검토 도구입니다. 원본과 추출 필드를 함께 확인하고, 규칙으로 찾은 불일치를 검토한 뒤 수정 내용과 판단을 기록해 검토 보고서로 내보냅니다.

- 두 서류를 업로드하고 필드 추출
- 각 값을 원본의 해당 줄과 대조해 확인
- 규칙 검사, 표시된 항목마다 판단 기록
- 모든 변경 이력, PDF 검토 보고서

프론트엔드	React · TypeScript · Vite
백엔드	NestJS · Zod · decimal.js
추출	Poppler · Tesseract · Gemini API
데이터	PostgreSQL · Redis · BullMQ

저장소: cargocheck-ai



합성 샘플 선적 건(SH-001)을 보여주는 개발 버전 작업 화면.

— CARGOCHECK AI · 추출

업로드부터 출처가 연결된 필드까지.

문제. 검토자는 송장과 포장명세서의 값을 직접 옮겨 적고 대조합니다. 자동 추출은 값을 잘못 읽을 수 있으므로, 모든 값은 원본 위치를 추적할 수 있어야 하고 사람이 확인해야 합니다.

1 업로드
송장과 포장명세서, 파일 시그니처 확인
`PDF · JPG · PNG`

→

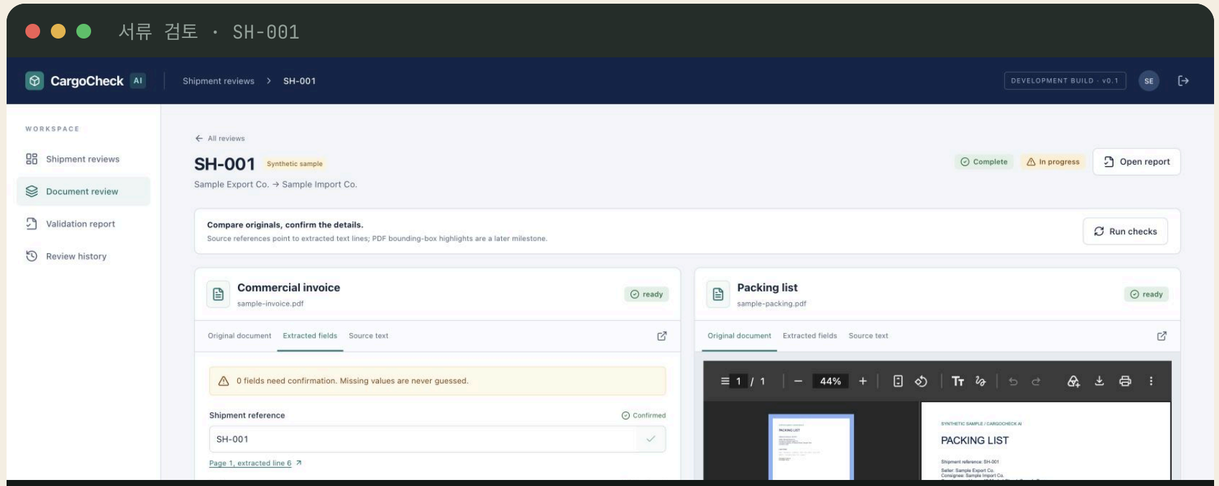
2 텍스트 읽기
텍스트 PDF는 Poppler, 스캔본과 이미지는 영어 Tesseract OCR로 처리
`pdftotext · tesseract -l eng`

→

3 파싱
로컬 모드: 보수적인 라벨 기반 템플릿 파서. 선택 모드: Gemini 구조화 추출
`EXTRACTION_PROVIDER=gemini`

→

✓ 확인
각 값은 페이지와 줄 정보를 유지하며, 검토자가 확인하거나 수정
`없는 값은 null 유지`



출처 참조가 있는 송장 추출 필드와 원본 문서를 함께 확인하는 화면. 이 샘플은 이미 확인된 상태입니다.

범용 AI가 아님

로컬 모드는 라벨이 있는 템플릿만 읽습니다. 찾지 못한 필드는 추측하지 않고 비워 둡니다.

모델 출력 검증

Gemini 모드의 응답은 Zod 스키마로 파싱하며, 검토자가 확인하기 전까지는 초안 값으로 취급합니다.

원본은 그대로

업로드한 파일은 비공개로 저장하고 변경하지 않습니다. 수정은 추출된 데이터에만 적용됩니다.

— CARGOCHECK AI · 검증

명시적인 규칙, 사람의 판단.

두 서류가 모두 확인되면 규칙으로 비교합니다. 규칙은 차이를 표시할 뿐이며, 그 의미와 이유는 검토자가 판단해 기록합니다.

— 비교 항목

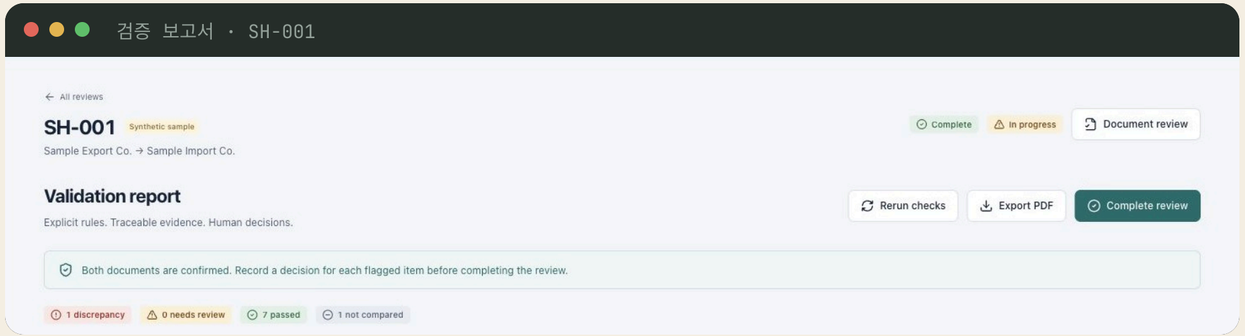
- 선적 참조번호, 판매자, 수하인, 주소
- 명시된 품목 코드, 모호한 매칭은 검토 대상으로 유지
- 호환되는 단위의 수량, 호환되지 않는 단위는 변환하지 않고 표시
- decimal.js로 송장 금액 계산: 수량 × 단가 = 기재 금액

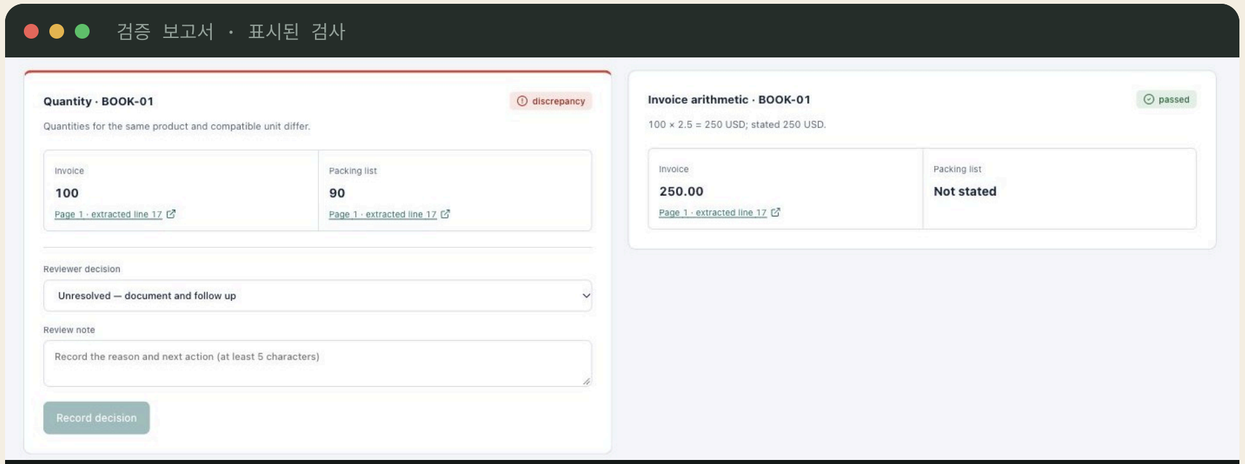
표시 항목마다 판단

표시된 모든 검사에 판단과 메모가 기록되어야 검토를 완료할 수 있습니다.

승인이 아님

검토 완료는 검토자의 작업을 기록할 뿐이며, 선적을 승인하지 않습니다.

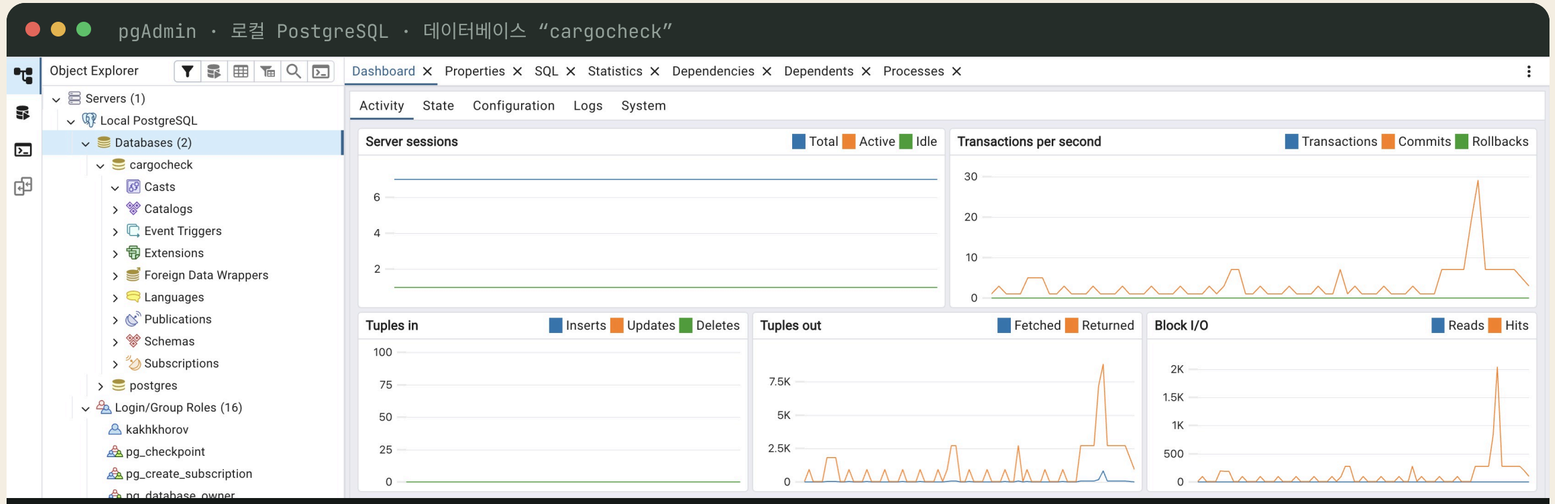




동일 품목의 송장 수량 100개와 포장 수량 90개가 달라 검토자가 판단을 기록해야 합니다. 100 × 2.50 = 250.00 USD는 통과합니다. 고객 데이터가 아닌 합성 샘플 값입니다.

— CARGOCHECK AI · 데이터 저장

PostgreSQL 어댑터가 실제로 저장하는 것.



CargoCheck AI 개발에 사용한 로컬 PostgreSQL 인스턴스의 pgAdmin 화면입니다. 테이블, 데이터 양, 성능 결과를 보여주는 화면은 아닙니다.

행 하나를 쓰는 이유

잠근 행 하나로 스키마 마이그레이션 없이 모든 쓰기를 일관되게 처리할 수 있어, 단일 인스턴스 개발 버전에 적합합니다.

트레이드오프

쓰기마다 전체 상태를 다시 저장하고 같은 잠금을 기다립니다. 운영 규모의 데이터량을 위한 구조는 아닙니다.

작업 큐

Redis를 설정하면 BullMQ 워커가 API 프로세스 안에서 동작합니다 (동시성 2, 백오프로 최대 3회 시도). Redis가 없으면 프로세스 내부에서 처리합니다.

cargo_state

PK	id	INTEGER
	data	JSONB NOT NULL

1번 행 → { users, sessions, shipments }

— 모든 변경은 하나의 트랜잭션으로

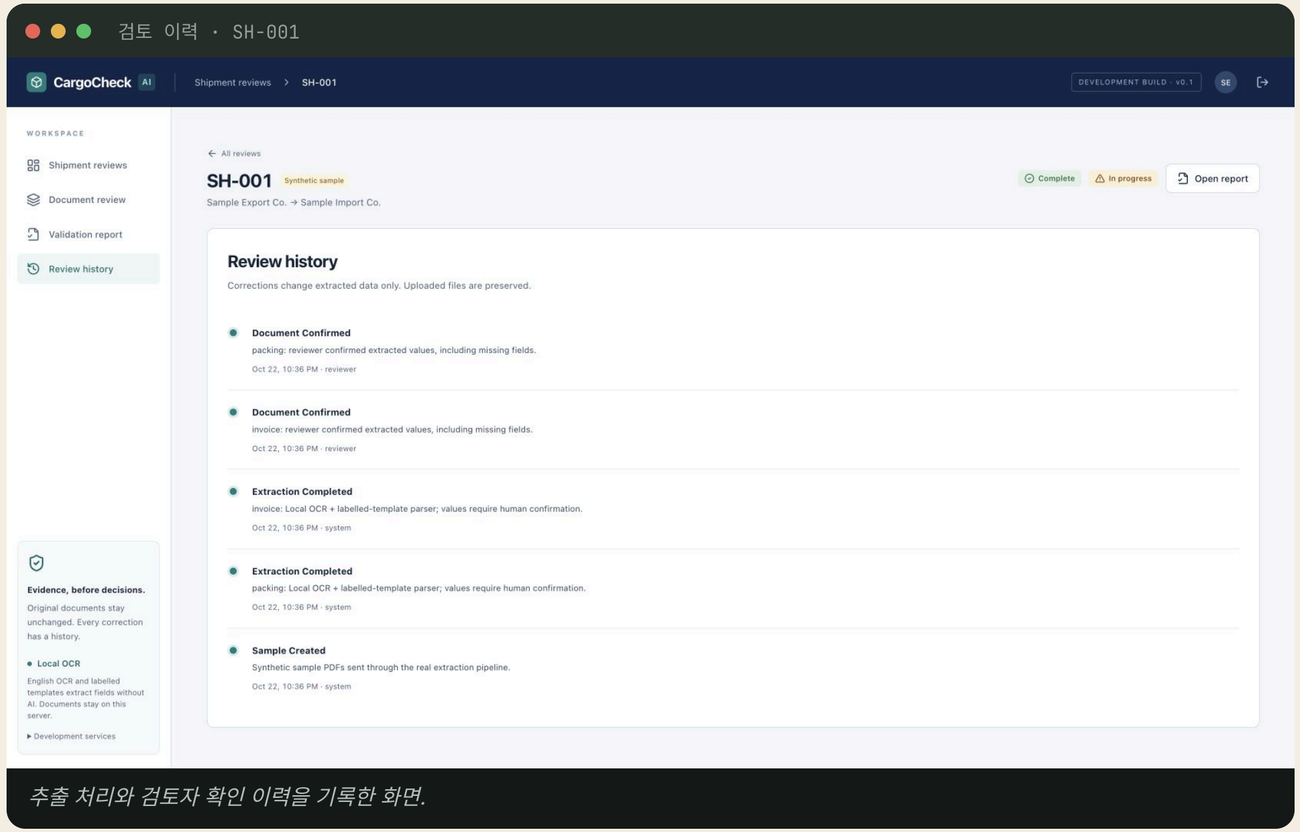
1. **BEGIN**
2. **SELECT data … FOR UPDATE**로 상태 행 잠금
3. **변경 적용** (애플리케이션 코드)
4. **UPDATE … COMMIT,** 오류 시 **ROLLBACK**

DATABASE_URL이 없으면 같은 코드가 로컬 JSON 파일에 저장합니다.

계획 단계(미구현): 선적, 서류, 필드, 검사, 판단을 위한 정규화 테이블과 마이그레이션.

— CARGOCHECK AI · 추적성

이력, 보고서, 검증 범위.



추출 처리와 검토자 확인 이력을 기록한 화면.

수정 시 다시 검토

수정은 원래 값을 보존하고 기존 판단을 무효로 표시하며, 완료된 검토를 다시 열 수 있습니다. 오래된 화면에서의 수정은 버전 충돌로 처리됩니다.

계정과 업로드

계정별 소유권 확인, HTTP-only 쿠키 세션, 솔트를 적용한 scrypt 해시, 업로드 시그니처 확인, 요청 제한, 변경 요청의 동일 출처 확인.

검토 보고서

PDF 내보내기는 현재 검사와 판단으로 검토 보고서를 만듭니다. 이 캡처에는 출력 파일이 포함되어 있지 않습니다.

검증

- ✓ 2026년 10월 22일 실행 중인 로컬 버전에서 화면 캡처
- ✓ 개발 과정에서 PostgreSQL·Redis 연동과 실제 Gemini 추출을 직접 테스트
- ✓ README에 자동화 테스트 11개 기록

아직 보여주지 않은 것

- ◌ Docker 기반 통합 테스트와 브라우저 QA
- ◌ 정확도, 속도, 비용 측정
- ◌ 공개 배포

— 주요 프로젝트 · 03

Lexara.

공동 프로젝트, 개발 참여. 다른 개발자와 함께 Lexara 개발에 참여했으며, 코드는 협업자의 GitHub 계정으로 반영했습니다. 제 저장소는 MakhmudSD/Lexara의 포크입니다.

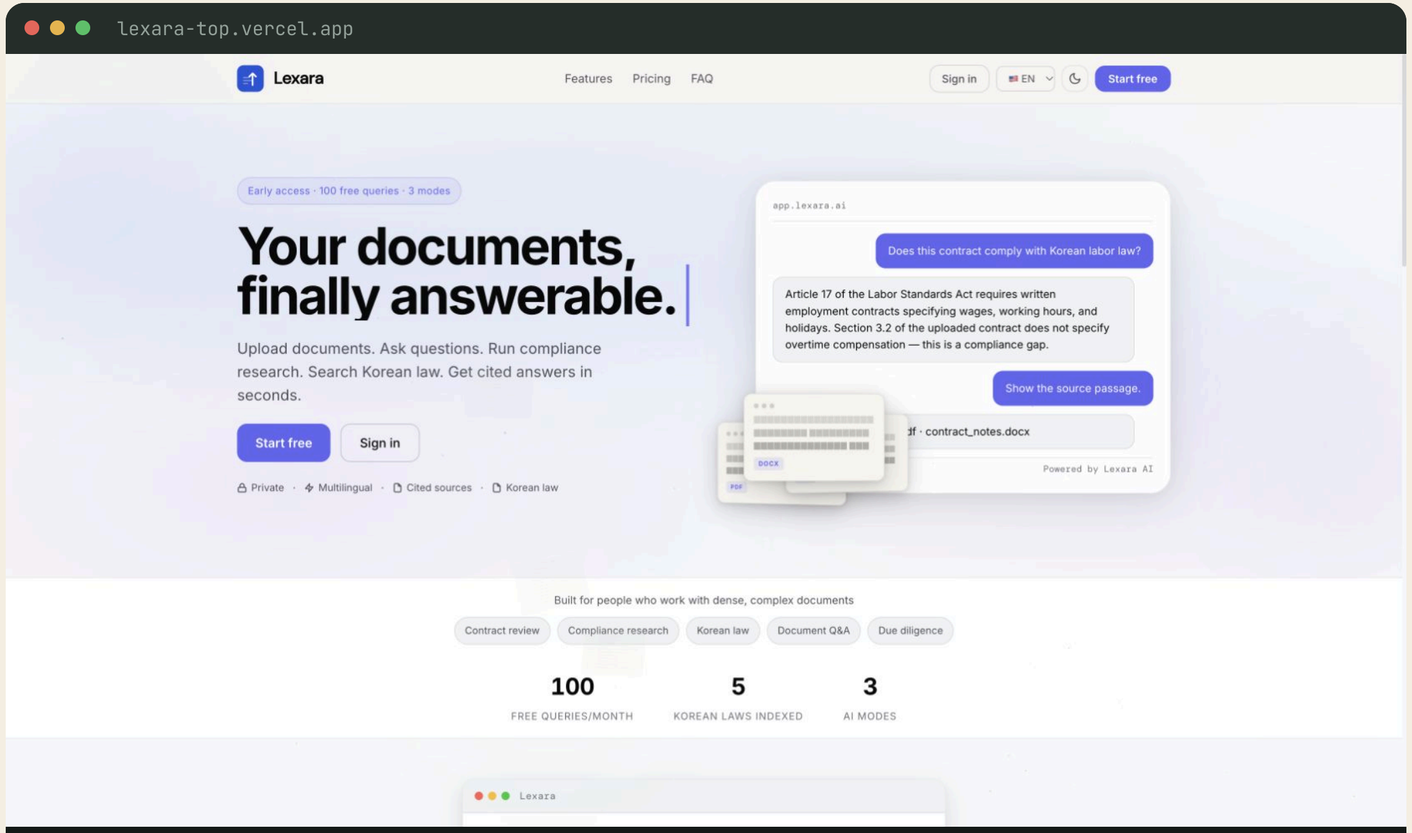
문서 질의응답 플랫폼입니다. 업로드한 문서를 작업 공간별로 관리하고, 답변에 근거가 된 문단을 출처로 표시합니다.

— 제품 아키텍처 (팀 개발, 현재 소스 기준)

- FastAPI, PostgreSQL, SQLAlchemy, Alembic
- pgvector 유사도 검색과 BM25를 RRF로 결합
- 업로드는 202를 반환하고 백그라운드 작업으로 처리, 상태 폴링 제공
- React/Vite 프론트엔드, OpenAI 모델, 선택적 R2 스토리지

공개 사이트 ↗

저장소: kakhkhorov97/Lexara (MakhmudSD/Lexara의 포크)



공동 프로젝트 Lexara의 공개 랜딩 페이지 (2026년 10월 22일). 화면 속 예시 답변은 데모 콘텐츠입니다.

이전 README는 FAISS를 설명하지만, 확인한 현재 소스는 pgvector를 사용합니다. 로그인, 업로드, 채팅은 이 포트폴리오를 위해 테스트하지 않았습니다.

— 주요 프로젝트 · 04 · 개발 중

Korea Visa Assistant

한국의 유학·구직 관련 체류자격 질문에 대해 공식 자료의 근거 문장을 찾는 한·영 검색 기반 어시스턴트입니다. 웹과 텔레그램에서 같은 엔진을 사용합니다.



진행 상황

핵심 코드, 웹·텔레그램 인터페이스, 테스트, 평가 도구가 있습니다. 자료 수집, 정답 라벨링, 배포는 진행 중입니다.

스크린샷 대신 다이어그램

아직 공개 데모가 없어, 소스 코드를 바탕으로 그린 아키텍처를 보여줍니다.

법률 자문이 아님

정보 검색 프로젝트입니다. 답변은 공식 자료를 가리키며, 사용자가 직접 확인해야 합니다.

저장소: korea-visa-rag

— CREATORHUB · 사례 연구

한국어 법령에 맞춘 검색.

문제. 한국 법령은 조 › 항 › 호 구조이고, 질문에는 D-10, E-7-1 같은 정확한 코드가 들어갑니다. 고정 길이 분할과 일반 토크나이저는 둘 다 쪼개 버립니다.

1
조문 단위 분할
제N조 또는 제N조의M마다 하나의 청크로 만들고 항과 호를 함께 유지

조문 표시 + 출처 URL

→

2
토큰화
Kiwi 형태소 분석, 비자 코드는 먼저 하나의 토큰으로 추출

D-10 · E-7-1

→

3
두 가지 검색
정확한 용어는 BM25, 의미는 Chroma 벡터로 검색

검색기별 후보 수

→

✓
결합과 인용
RRF로 두 목록을 결합하고, 답변에 사용한 문단을 인용

RRF_K

출처 기반 답변
프롬프트는 검색된 출처만 사용하게 하고, 출처로 답할 수 없으면 그렇다고 말하도록 합니다. 설계상의 선택이며 정확성을 증명하지는 않습니다.

엔진 하나, 인터페이스 둘
FastAPI 웹과 텔레그램 봇이 API 키 없는 검색 전용 모드를 포함해 같은 어시스턴트를 호출합니다.

평가 계획
hit@k, recall@k, MRR을 구현했습니다. 시작용 질문 12개는 정답 라벨이 아직 없으며, 공개한 결과는 없습니다.

Python · FastAPI · Kiwi · rank_bm25 · Chroma · OpenAI API · Telegram Bot API · pytest · GitHub Actions

— 이전 프로젝트

Sofiya Bookstore

Express REST API와 MongoDB 기반의 React·Redux Toolkit 스토어프론트입니다. 독자는 도서를 둘러보고 장바구니에 담아 주문을 조회하며, EJS 관리자 패널로 카탈로그를 관리합니다. 모든 주문 조회는 로그인한 회원 기준으로 필터링합니다. 저장소: Sofiya-react · Sofiya-Bookstore

장바구니
책과 수량 선택

→

주문 저장
주문 항목 저장, 100 이상이면 배송비 무료, 미만이면 5

→

보류
독자의 확인 대기

→

처리 중
확인 완료, 회원 포인트 적립

→

배송 완료
주문 완료

관리자 패널

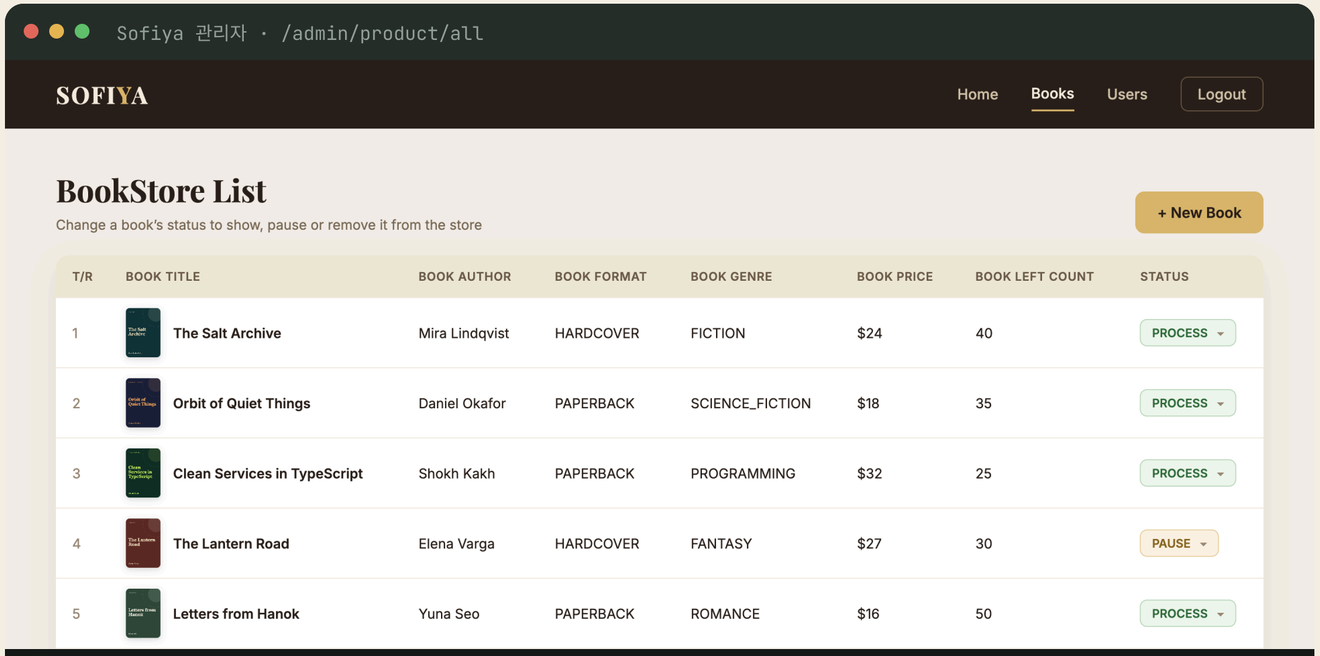
서버에서 렌더링하는 EJS 페이지: 로그인, 상품 목록, 이미지 최대 5장의 신규 상품 등록, 상품 수정, 회원 상태를 바꿀 수 있는 회원 목록.

두 가지 로그인 경로

독자는 24시간 유효한 JWT를 쿠키로 받고, 모든 API 호출에서 검증됩니다. 관리자 패널은 MongoDB에 저장되는 6시간 세션을 쓰며, 스토어 계정만 가드를 통과합니다.

지금이라면 바꿀 점

상품 가격이 브라우저에서 넘어오고 결제 단계는 목업입니다. 지금이라면 서버에서 합계를 다시 계산하고 실제 결제 서비스를 연동하겠습니다.



관리자 도서 목록 페이지를 EJS 템플릿을 바탕으로 데모 도서와 함께 다시 그린 그림입니다. 배포되지 않은 프로젝트라 실제 화면 캡처가 아닙니다.

— 연락처

아이디어부터 운영까지

이메일 kakhkhorov97@gmail.com

전화 010-7901-7555

카카오톡 kakhkhorov_A

GITHUB github.com/kakhkhorov97

라이브 프로젝트 creatorhub.site

주소 대한민국 경기도 수원시

라이브 프로젝트
creatorhub.site

GitHub
github.com/kakhkhorov97

